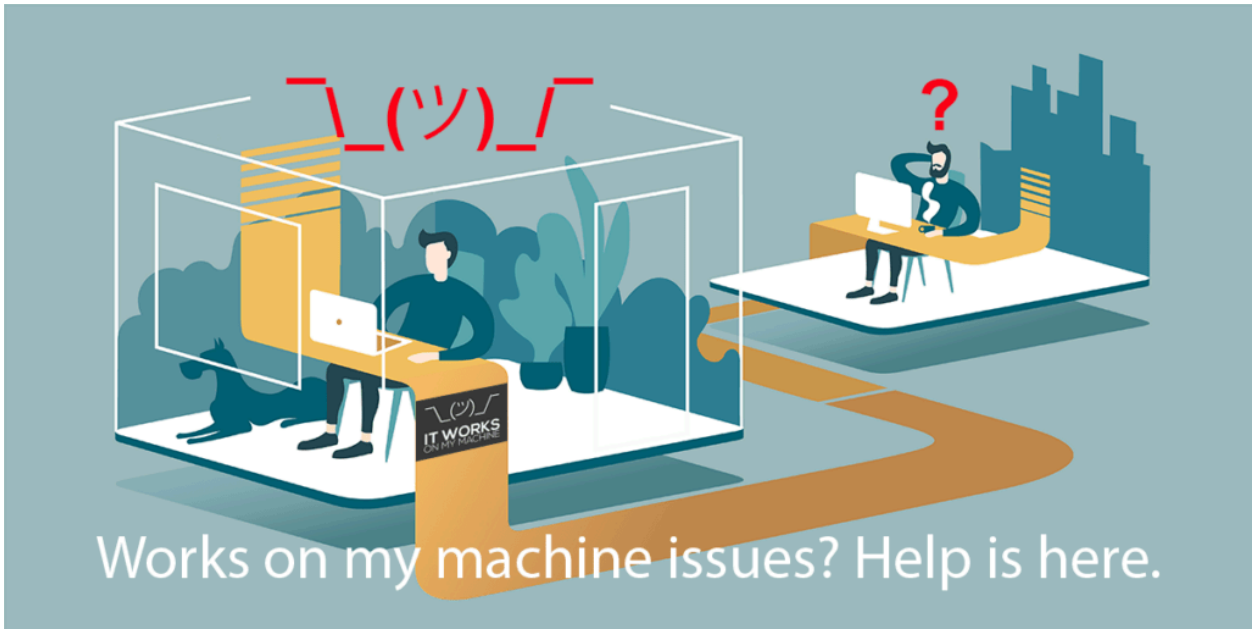
National University of Computer and Emerging Sciences Islamabad Operating System Lab Spring 2025

Lab Manual: Docker

This lab manual provides a step-by-step guide to setting up and using Docker on Ubuntu. The process includes installing Docker, creating a simple project with Dockerfiles, and pushing an image to Docker Hub



Why Docker?

Imagine you build a C++ application on your machine, but it doesn't run on your friend's machine because their system is missing some libraries. Docker solves this by **packing your app and all its dependencies into a container** so it works anywhere—on your machine, your friend's, or even on a cloud server.

Docker vs Virtual Machine (VM)

Feature	Docker	Virtual Machine
Environment	Shares host OS kernel	Full OS for each VM
Speed	Starts in seconds	Slower startup (minutes)
Size	Lightweight (MBs)	Heavy (GBs)
Performance	Faster, less overhead	Slower, more resource-intensive
Use Case	Running individual apps in isolation	Running multiple full OS environments

Part 1: Installing Docker

Step 1: Update Package Lists

Update the package lists for Ubuntu to ensure you have the latest version of packages.

sudo apt-get update

- Open a terminal and type the above command.
- Wait for the update process to complete.

```
perating-system-lab@operating-system-lab-VirtualBox:~$ sudo apt-get update
sudo] password for operating-system-lab:
it:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
it:2 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
it:3 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
et:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
et:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [588
B]
et:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [7,
28 B]
et:7 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Componen
s [212 B]
et:8 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [
00 kB]
et:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components
[52.0 kB]
et:10 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Compone
ts [208 B]
etched 1,574 kB in 6s (248 kB/s)
eading package lists... Done
```

Step 2: Install Docker

Install Docker from the default Ubuntu repositories.

sudo apt install docker.io

- Confirm the installation by typing Y when prompted.
- Wait for the installation process to complete.

```
Try: sudo apt install <deb name>
operating-system-lab@operating-system-lab-VirtualBox:~$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd git git-man liberror-perl pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  bridge-utils containerd docker.io git git-man liberror-perl pigz runc
  ubuntu-fan
0 upgraded, 9 newly installed, 0 to remove and 7 not upgraded.
Need to get 81.2 MB of archives.
After this operation, 312 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9
Get:3 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.1.12-0ubuntu3.1 [8,
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 git-man all 1:2.43.0-1ubuntu7.2 [
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 git amd64 1:2.43.0-1ubuntu7.2 [3,
Get:6 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 1.7.12-0ubuntu4
Get:7 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 24.0.7-0ubun
87% [7 docker.io 22.7 MB/29.1 MB 78%]                                    87% [7 docker.io 23
Get:8 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 liberror-perl all 0.17029-2 [25.6 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 ubuntu-fan all 0.12.16 [35.2 kB]
Fetched 81.2 MB in 5min 50s (232 kB/s)
Preconfiguring packages ...
```

Step 3: Install Docker Using Snap (Optional)

For an alternative installation method, use Snap.

sudo snap install docker

```
Processing triggers for man-db (2.12.0-4build2) ...
operating-system-lab@operating-system-lab-VirtualBox:~$ sudo snap install docker
Download snap "docker" (2963) from channel "stable"
             11%  175kB/s 12.3m
docker 27.2.0 from Canonical✓ installed
operating-system-lab@operating-system-lab-VirtualBox:~$
```

- Verify the Snap version of Docker is installed.

Step 4: Check Docker Version

Ensure Docker is installed correctly by checking its version.

sudo docker --version

```
              11%  175kB/s 12.3M
docker 27.2.0 from Canonical✓ installed
operating-system-lab@operating-system-lab-VirtualBox:~$ sudo docker --version
Docker version 24.0.7, build 24.0.7-0ubuntu4.1
operating-system-lab@operating-system-lab-VirtualBox:~$
```

Step 5: Test Docker Installation

Run the Hello World container to verify that Docker is functioning properly.

sudo docker run hello-world

```
operating-system-lab@operating-system-lab-VirtualBox:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:d715f14f9eca81473d9112df50457893aa4d099adeb4729f679006bf5ea12407
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

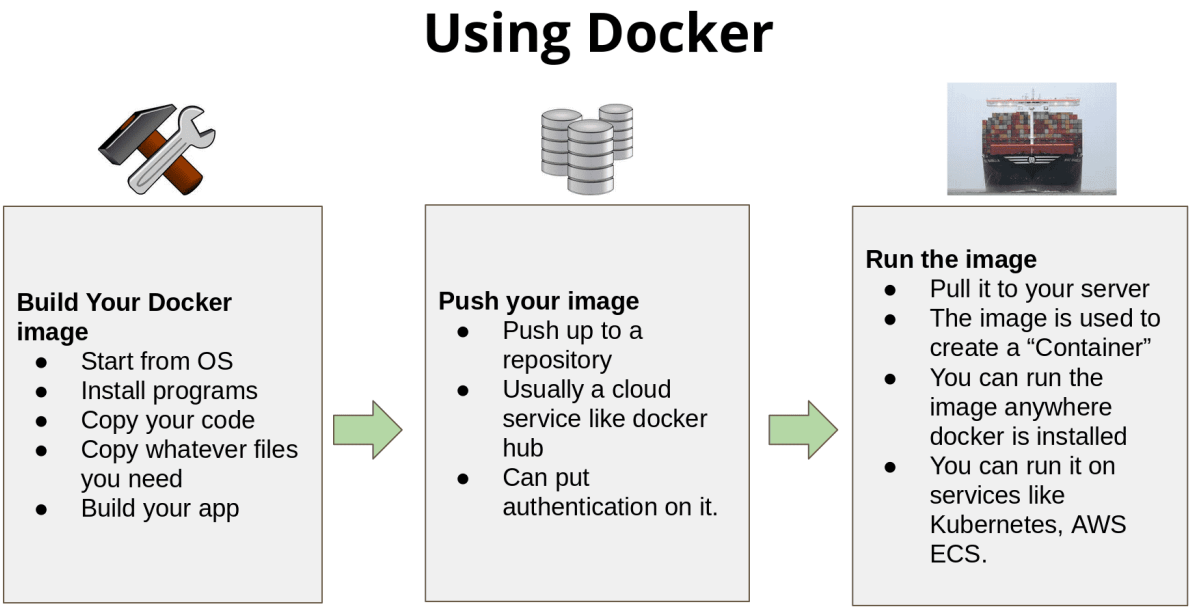
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/
```

- You should see a message confirming that Docker is installed and running correctly.

Part 2: Setting Up a Docker Project



Step 1: Create a Project Folder

Create a folder on the desktop for your Docker project.

cd ~/Desktop

mkdir docker

cd docker

```
operating-system-lab@operating-system-lab-VirtualBox:~$ cd Desktop
operating-system-lab@operating-system-lab-VirtualBox:~/Desktop$ mkdir docker
operating-system-lab@operating-system-lab-VirtualBox:~/Desktop$ cd docker
operating-system-lab@operating-system-lab-VirtualBox:~/Desktop/docker$
```

- Navigate to your Desktop.
- Create a folder named `docker` and move into it.

Step 2: Create Required Files

Create two files in the `docker` folder: `Dockerfile` and `main.cpp`.

touch Dockerfile

touch main.cpp

```
operating-system-lab@operating-system-lab-VirtualBox:~/Desktop/docker$ touch Dockerfile
operating-system-lab@operating-system-lab-VirtualBox:~/Desktop/docker$ touch main.cpp
operating-system-lab@operating-system-lab-VirtualBox:~/Desktop/docker$
```

- Use a text editor (e.g., `nano` or `vim`) to edit these files.

Step 3: Write Code for Dockerfile

Edit the `Dockerfile` to include the following content:

```
FROM gcc:latest
COPY . /usr/src/cpp_test
WORKDIR /usr/src/cpp_test
RUN g++ -o test main.cpp
CMD ["./test"]
```

- This file specifies the base image (`gcc:latest`), sets up the working directory, compiles the C++ code, and defines the command to execute.

Step 4: Write Code for `main.cpp`

Edit the `main.cpp` file to include a simple C++ program. For example:

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, Docker!" << std::endl;
    return 0;
}
```

- Save and close the file.

Part 3: Working with Docker Hub

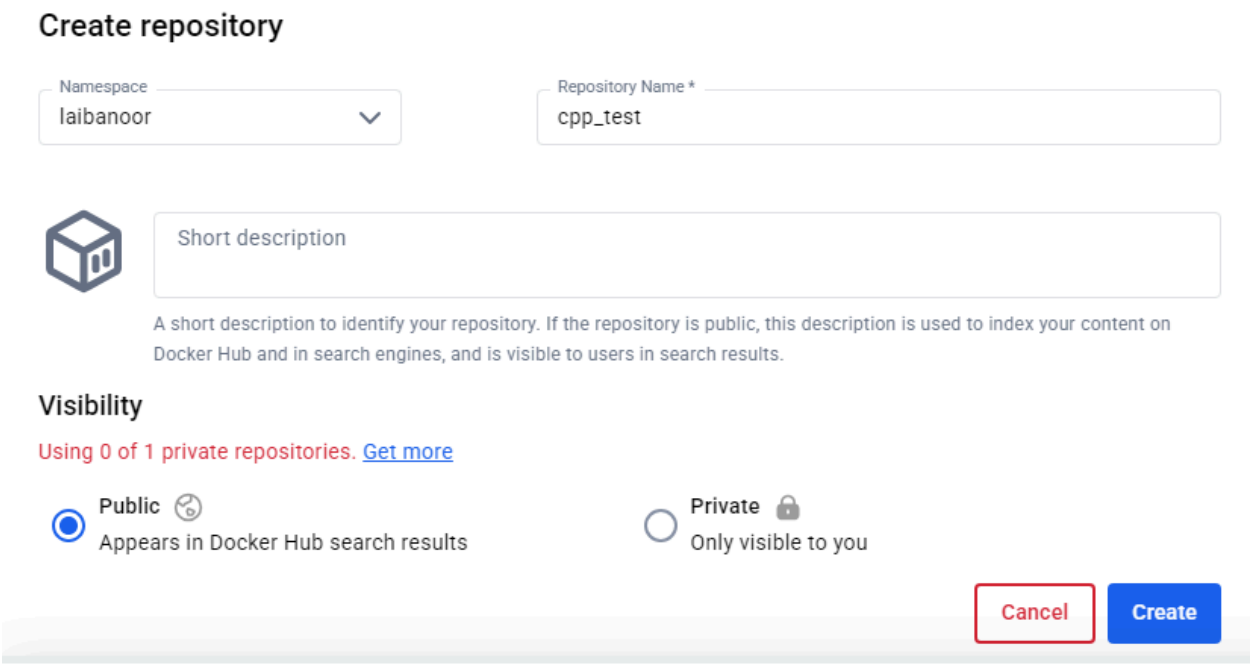
Step 1: Create a Docker Hub Account

1. Go to the Docker Hub website.
2. Click on **Sign Up** in the top-right corner.
3. Fill in the registration form with the following details:
    - **Username**: Choose a unique username for your Docker account.
    - **Email Address**: Provide a valid email address.
    - **Password**: Create a strong password.
4. Agree to the terms of service and click **Sign Up**.
5. Verify your email address by clicking the link sent to your email.
6. Once verified, log in to your Docker Hub account.

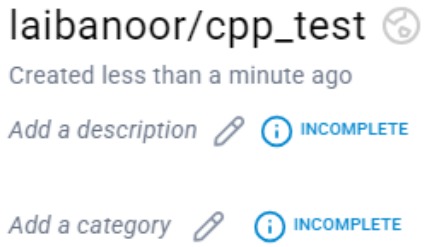
Step 2: Create a Repository on Docker Hub

1. After logging in to your Docker Hub account, click on **Repositories** in the navigation bar.
2. Click the **Create Repository** button.

3. Fill in the repository details:
    - **Repository Name**: Provide a name for your repository (e.g., cpp_test).
    - **Description** (optional): Add a brief description of the repository.
    - **Visibility**: Choose between public or private visibility.
4. Click **Create** to finalize the repository creation.



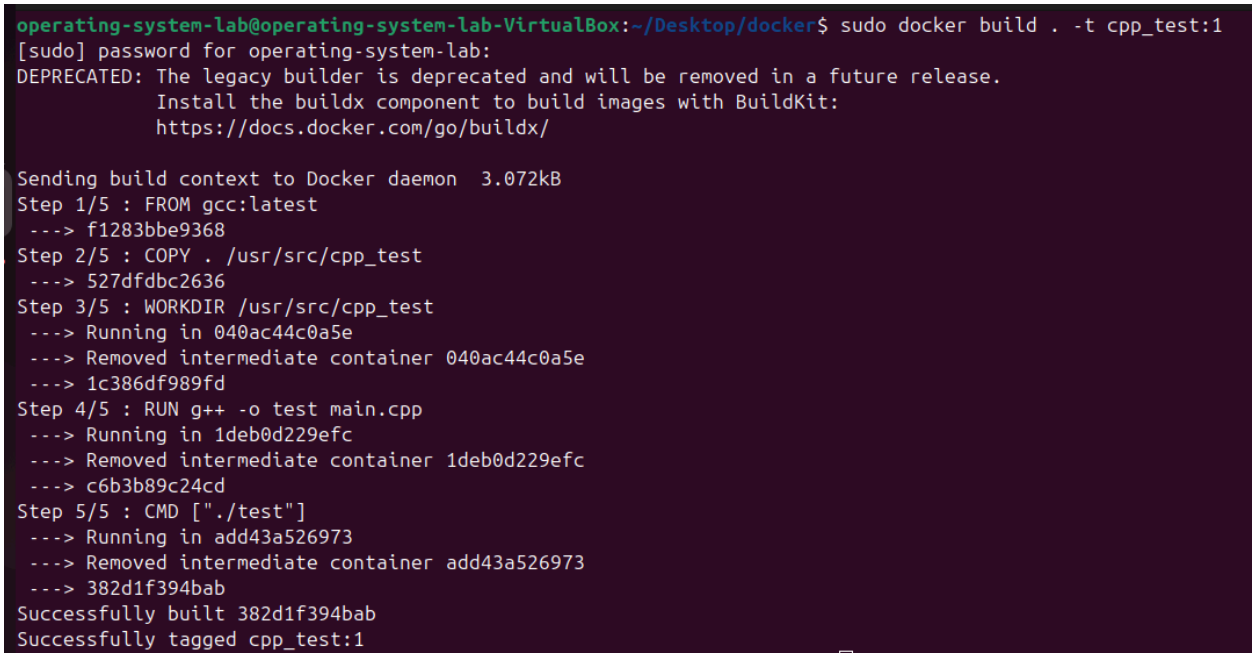
5. Your new repository will now be visible on your Docker Hub dashboard.



Step 3: Build the Docker Image

Build a Docker image for the C++ project.

sudo docker build . -t cpp_test:1

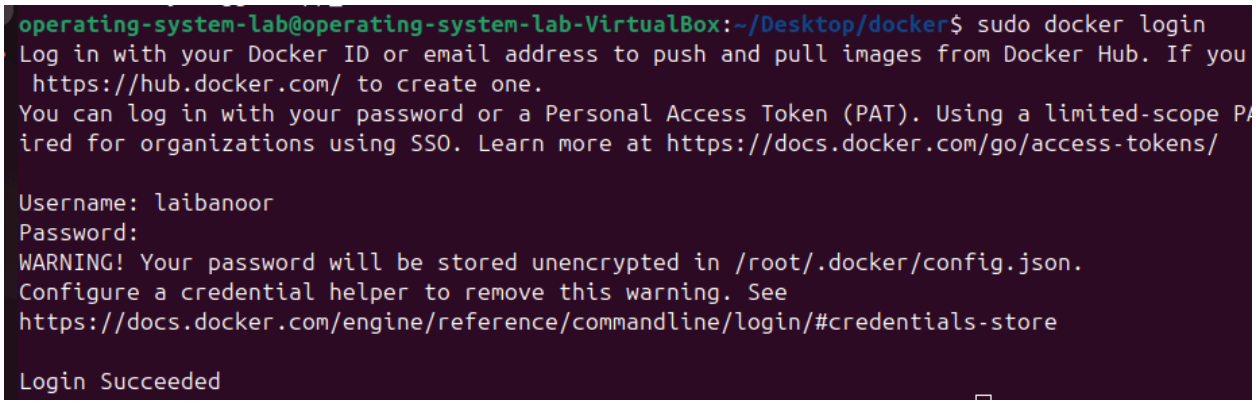


- The `-t` flag tags the image with the name `cpp_test` and version `1`.

Step 4: Log in to Docker Hub

Log in to your Docker Hub account.

sudo docker login



- Enter your Docker Hub username and password when prompted.

Step 5: Tag the Docker Image

Tag the Docker image with your Docker Hub repository name.

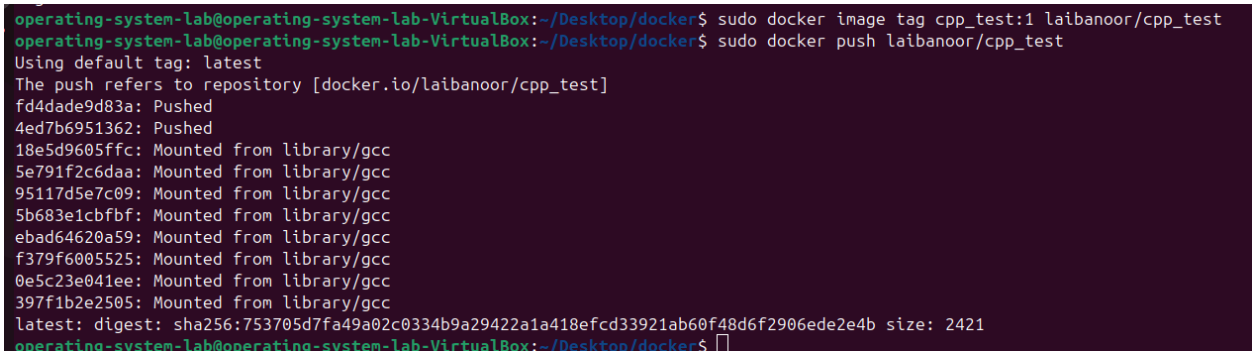
sudo docker image tag cpp_test:1 username/repositoryname

- Replace `username` with your Docker Hub username.
- Replace `repositoryname` with the name of your repository.

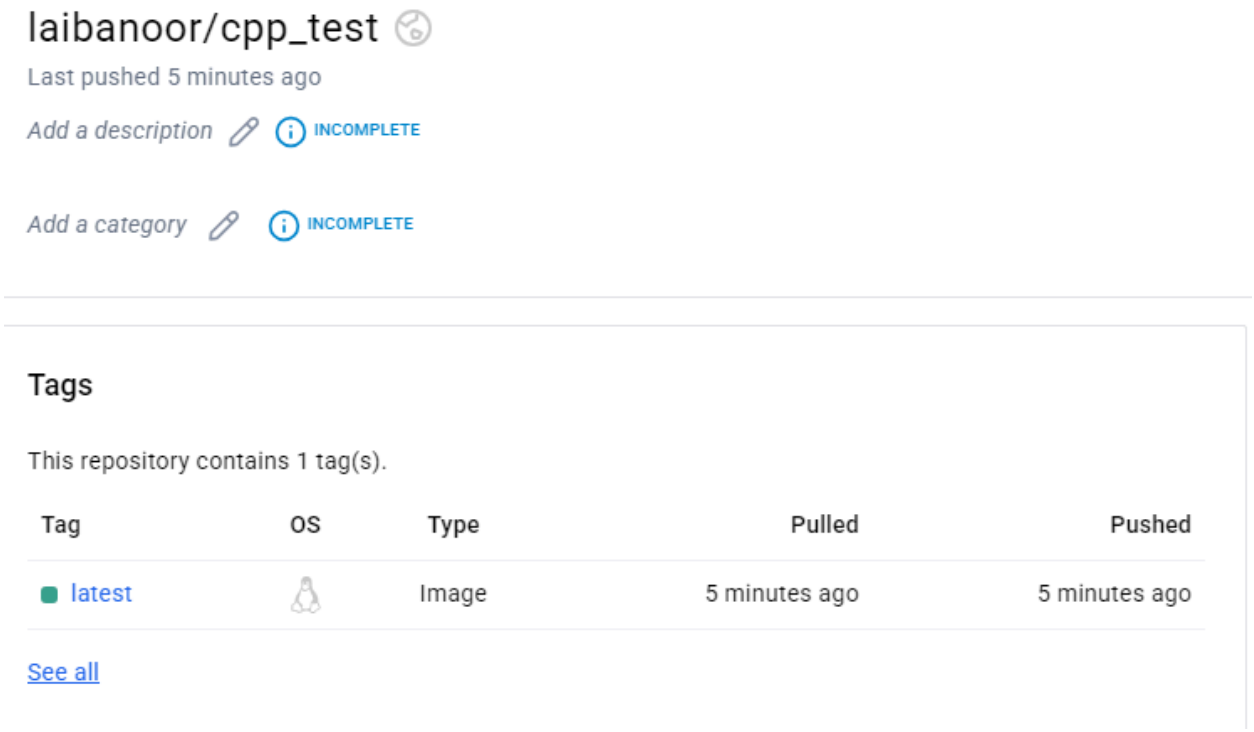
Step 6: Push the Image to Docker Hub

Push the Docker image to your Docker Hub repository.

sudo docker push username/repositoryname



- Replace `username` and `repositoryname` with your Docker Hub details.
- Verify that the image appears in your Docker Hub repository.



Conclusion

By following this manual, you have successfully installed Docker on Ubuntu, created a simple Docker project, and pushed the image to Docker Hub.

THE END
